SemSched

Bottleneck-Aware Scheduling via Semantic Resource Profiles

Research Group Meeting — March 2026

Target: EuroSys / SoCC 2027

The Problem

Why current cluster schedulers fail for elastic, multi-phase workloads

Agents Are a New Kind of Workload

What Agents Do

AI coding agents (Claude Code, Devin, OpenHands) orchestrate **multi-phase workloads** on shared clusters:

1. Read & understand codebase
2. Edit source files
3. Compile (Go, Rust, C++)
4. Run tests / benchmarks
5. Iterate on failures

Each phase has **different** resource needs

What Schedulers See

Traditional schedulers (SLURM, YARN, K8s) see only:

```yaml
resources:
  cpu: 4
  memory: 8Gi
```

- No bottleneck type (CPU? Memory? I/O?)
- No elasticity (does adding nodes help?)
- No phase structure (what comes next?)
- No self-description of any kind

> **Result:** Schedulers allocate blindly, wasting 20-30% of cluster makespan

Three Mismatches

Bottleneck Mismatch

A **memory-bound** KV store and a **CPU-bound** ML trainer both request "4 nodes"

Extra node for the ML trainer saves **10x more time** than for the KV store

Scheduler can't tell the difference

Temporal Mismatch

Agent switches from **compile** (CPU-heavy) to **test** (I/O-heavy)

Scheduler reacts **after** the phase change — too late

With phase-ahead: scheduler **pre-allocates** before transition

Elasticity Mismatch

Redis peaks at 2 cores (memory-bound)

Spark scales to 32 cores (parallelizable)

Without elasticity curves, scheduler **over-provisions** Redis, **under-provisions** Spark

All three are solvable if the workload declares its needs

Our Approach: Semantic Resource Profiles

A new scheduling abstraction

The Semantic Profile

Workloads declare a **semantic resource profile** to the scheduler:

```
SemanticProfile(
  # Elasticity
  min_nodes=2,
  preferred_nodes=8,
  max_nodes=16,

  # Bottleneck type
  bottleneck="cpu",  # or memory, io, network

  # Phase DAG
  phases=[
    Phase("compile", duration=300,
          bottleneck="cpu", nodes=8),
    Phase("test", duration=120,
          bottleneck="io", nodes=4),
    Phase("benchmark", duration=60,
          bottleneck="memory", nodes=2),
  ],

  # Scaling curve
  elasticity=amdahl(parallel_fraction=0.85)
)
```

Why This Works

Agents are natural producers of these profiles:

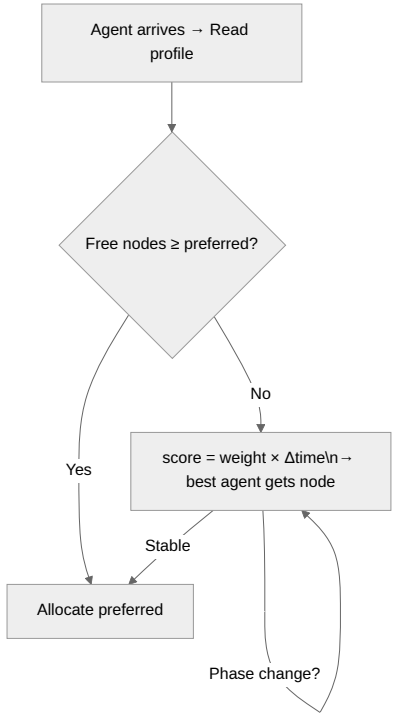
- They plan workloads before executing
- They understand algorithm characteristics
- They can adapt to scheduler feedback

What's New

No existing system combines:

1. **Declared** elastic profiles (vs inferred)
2. **Bottleneck-type** awareness
3. **Phase-ahead** scheduling
4. For **exclusive** cluster resources

SemSched: The Algorithm



Bottleneck Weights

Type	Weight	Why
CPU	1.0	Near-linear scaling
I/O	0.8	Good parallelism
Network	0.5	Moderate benefit
Memory	0.2	Extra nodes rarely help

Key idea: Under contention, give next node to the agent whose bottleneck type benefits most from extra resources.

Phase-ahead: read upcoming phases from profile, pre-allocate before transition.

Why Bottleneck Weighting Matters

Without bottleneck awareness

```
Agent A (memory-bound): 4 nodes → 5 nodes
  Time saved: ~1.2 seconds

Agent B (CPU-bound): 4 nodes → 5 nodes
  Time saved: ~12.5 seconds
```

Equal allocation: give node to A or B with equal probability

Expected savings: ~6.85s

With bottleneck awareness

```
Agent A (memory, weight=0.2):
  Score = 0.2 × 1.2 = 0.24

Agent B (CPU, weight=1.0):
  Score = 1.0 × 12.5 = 12.5
```

Always give node to B first

Expected savings: 12.5s (1.8x better)

> **Design decision (D7):** Pure `bottleneck_weight × marginal_savings` outperforms adding a work-remaining factor. Simpler, more principled, and produces 4-14% makespan improvement.

Simulation Results

4 schedulers, 4 scenarios, 16 nodes

Experimental Setup

Schedulers

1. **FIFO** — first-come, first-served, pack to preferred
2. **Static** — equal partition across active agents
3. **Round-robin** — cycle through agents, one node each
4. **SemSched** — bottleneck-weighted marginal gain + phase-ahead

Workloads (Elasticity Models)

- **kv_store** (Redis): memory-bound, flat scaling (Amdahl p=0.3)
- **analytics** (Spark TPC-H): I/O-bound, moderate scaling (p=0.7)
- **ml_train** (PyTorch): CPU-bound, near-linear scaling (p=0.95)

Scenarios

Scenario	Agents	Pressure
Basic	3	Low (min=6/16)
Staggered	5	Medium, sequential
Burst	8	High, simultaneous
Stress	8	Oversubscribed (min=19>16)

Simulator

- Discrete-event engine (`sim/engine.py`)
- No wasted timesteps — event-driven
- Deterministic: scripted agents, no LLM variance

Results: SemSched Wins in All Scenarios

Scenario	FIFO	Static	RoundRobin	**SemSched**	**vs FIFO**	**vs Static**
Basic (3)	660	993	990	**588**	**+11.0%**	**+40.8%**
Staggered (5)	836	918	1189	**720**	**+13.8%**	**+21.5%**
Burst (8)	1480	1790	2360	**1415**	**+4.4%**	**+20.9%**
Stress (8)	1620	1395	2420	**1442**	**+11.0%**	**-3.2%**

> **+4.4–13.8%** vs FIFO across all scenarios | **+20.9–40.8%** vs static in 3/4 | Only loses to static under extreme oversubscription (min=19 > 16 nodes)

Real Elasticity Measurements

Measured on real hardware (gpu08/gpu10) to calibrate simulation:

Redis (Memory-Bound)

- Peaks at **2 cores** (1.6x speedup)
- **Degrades severely** at 128 cores (0.4x)
- Validates weight=0.2 for memory-bound

Spark TPC-H (I/O-Bound)

- Scales to **32 cores** (10.7x)
- Degrades at 64-128 (shuffle overhead)
- Validates weight=0.8 for I/O-bound

PyTorch ResNet-50 (CPU-Bound)

- Scales to **64 cores** (22.7x)
- NUMA wall at 128 cores (18.3x)
- Validates weight=1.0 for CPU-bound

LLVM Build (CPU-Bound)

- **Near-linear** to 128 cores (28.8x)
- Pure parallelism, no data dependencies
- Strong validation of Amdahl's model

Data in `experiments/elasticity/` — real measurements confirm our bottleneck weight assignments

Competitive Landscape

No existing system occupies our niche

Positioning

Feature	SLURM/K8s	Pollux/Sia	SchedMate	AgentCgroup	**SemSched**
Self-describes	No	No (profiled)	No (LLM)	Hints	**Yes (full)**
Elastic range	No	Yes (observed)	No	No	**Yes (declared)**
Bottleneck-aware	No	No	No	Partial	**Yes (weighted)**
Phase-ahead	No	No	No	No	**Yes**
Agent-cooperative	No	No	No	Partial	**Yes**
General workloads	Yes	**No**	Yes	Yes	**Yes**

vs Pollux/Sia/Lyra: They infer elasticity by observing throughput. We use declared profiles — enables phase-ahead and works for non-DL workloads.

vs SchedMate: They infer semantics via LLM from job descriptions. We use cooperative declaration — richer, more reliable, no inference errors.

Related Work Validates Our Thesis

AgentCgroup (arXiv:2602.09345)

Characterized AI agent OS-level resources:

- **15.4x** peak-to-average memory ratio
- Memory (not CPU) is the bottleneck
- Demands are **highly unpredictable** from outside
- 1.8x variance across runs of same task

Their conclusion supports ours: agents should declare needs proactively, because external prediction fails.

We reproduced their trace analysis — numbers confirmed.

Key Numbers from Reproduction

Metric	Paper	Ours
Tool execution share	42.5%	42.5%
LLM thinking share	57.5%	57.5%
CPU utilization	<13% avg	~12%
Framework baseline	~185 MB	~180 MB

Reproduced from their open-source traces (33 Haiku tasks). Two-layer memory structure and burst-silence pattern confirmed.

Workload Characterization

Building realistic test workloads

Perf-bench: Real Performance Issues as Agent Workloads

SWE-bench-style Collection

1. **Label discovery** on GitHub repos
2. **Scrape** closed perf-labeled issues
3. **Link** to merged fixing PRs
4. **3-way patch split**: code / test / benchmark
5. **Checkout base_commit** = broken state for agent

Go/Rust builds are **resource-intensive** — ideal for scheduling research

Agent Runner

Blind Claude Code agent per instance, CPU/mem @ 1s intervals

Collection Results (40 instances)

Repo	N	Tests	Bench
etcd-io/etcd (Go)	10	4	0
tikv/tikv (Rust)	13	0	0
cockroachdb/cockroach (Go)	17	8	2
Total	**40**	**12**	**2**

Traces calibrate simulation elasticity curves — replacing synthetic models with real agent resource profiles

Paper Status & Plan

EuroSys / SoCC 2027 submission

Paper Progress

Written

- **Abstract** (~400 words)
- **Section 1: Introduction** (~2000 words)
  - Agent workloads as motivation
  - Three mismatches
  - Contribution statement
- **Section 2: Background** (~3800 words)
  - Agents as infrastructure users
  - Why existing schedulers fail
  - Three motivating scenarios
  - Table 1: scheduler landscape
- **Bibliography**: 36 references curated

To Write

- **Section 3: Design** — profile formalization
- **Section 4: Algorithm** — SemSched details
- **Section 5: Implementation** — simulator + workloads
- **Section 6: Evaluation**
  - Simulation: 16/32/64/128/256 nodes
  - Real cluster: 16 nodes, ablations
  - Comparison with Pollux/Tiresias (via Blox)
- **Section 7: Related Work** (survey done)
- **Section 8: Conclusion**

Paper at `~/latex-pro/SemanticSched_EuroSys/` — ACM sigplan twocolumn format

What’s Left to Do

Evaluation Gaps

- Ablation variants (semantic-no-bottleneck, etc.)
- Scale test: 64/128/256 nodes, 20-50 agents
- Blox integration for Pollux/Tiresias baselines
- Real cluster validation (16 nodes, Redis+Spark+PyTorch)

Workload Calibration

- Replace abstract elasticity with Amdahl’s law fits
- Run perf-bench agents, collect resource traces
- Compare traces vs AgentCgroup SWE-bench traces
- Validate burst-silence patterns in Go/Rust workloads

Paper Writing

- Sections 3-8 (design through conclusion)
- Figures: elasticity curves, timeline plots, allocation heatmaps
- Frame as **general abstraction**, not just "agent scheduling"

Venue Strategy

Venue	Deadline	Fit	Notes
SoCC 2027	~Jun 2027	Best	Cloud + scheduling
EuroSys 2027	~Oct 2026	Stretch	Needs real cluster eval
ATC 2027	~Jan 2027	Good	Practical systems
Middleware 2027	~May 2027	Safe	Middleware + scheduling

Key Framing Decision (D5, Resolved)

General "semantic resource profiles" — NOT "AI agent scheduling"

- Broader audience, not tied to AI hype cycle
- Agents are the motivating use case, but the abstraction works for any self-aware workload

Discussion Questions

1. Evaluation Priorities

What do reviewers care most about? More simulation scale (256 nodes, 50 agents)? Or real cluster validation (16 nodes, 3 real workloads)?

2. Ablation Design

Which ablations are most convincing? No-bottleneck vs no-phase vs no-rebalance? Or comparison against Pollux/Tiresias via Blox?

3. Workload Calibration

Is the perf-bench approach (real GitHub issues) sufficient for workload characterization, or do we also need controlled microbenchmarks?

4. Venue Timing

Thank You

Simulation: `semantic-scheduling/` — 4 schedulers, 4 scenarios, working

Paper: `~/latex-pro/SemanticSched_EuroSys/` — intro + background written

Workloads: `perf-bench/instances/` — 40 real perf issues collected

Traces: `agentcgroup/experiments/` — AgentCgroup reproduction confirmed

Key references: AgentCgroup (arXiv:2602.09345) · SchedMate (arXiv:2510.03334) · Pollux (OSDI'21) · Sia (SOSP'23) · SWE-bench (ICLR'24)